Auditoría de código al grupo par (api_winbin)

Auditor: Oswal Alexander Wscanga Bandres (rol QA). Fecha: 17/09/2026.

Repositorio revisado: https://github.com/jhongaleano/api_winbin (rama master). Es el proyecto de nuestros compañeros, el grupo al que le tocó evaluar su entrega del Sprint 1.

1. Procedimiento de auditoría ejecutado

Clonamos el repositorio desde GitHub y revisamos su estructura (backend en Java con Spring Boot y Gradle). Compilamos el proyecto con Java 21 y la compilación pasó sin errores. Después levantamos la aplicación de forma local en el puerto 8080 conectada a una base de datos MySQL para probar los endpoints reales y revisamos el código fuente para confirmar cada hallazgo.

2. Resumen de los hallazgos

Enviamos el reporte con tres hallazgos principales, uno por cada categoría que pedía la actividad (seguridad, lógica y normalización).

#	Hallazgo	Categoría	Severidad
H1	Las contraseñas de los usuarios quedan expuestas en las respuestas de la API	Error de seguridad	Crítica
H2	El sistema de puntos y ranking nunca suma puntos	Bug de lógica	Alta
H3	El total de puntos del curso no se mantiene al día	Falla de normalización	Media

3. H1 — Las contraseñas de los usuarios quedan expuestas (severidad: crítica)

El problema más grave que encontramos. La API devuelve el objeto completo de *usuarios*, y ese objeto incluye el campo *contrasenna*, donde se guarda la contraseña cifrada con BCrypt. No hay ningún DTO ni *@JsonIgnore* que lo oculte.

Lo comprobamos en dos rutas que son públicas (no piden token):

- *POST /api/usuarios/registro* (registro de usuarios): en *usuarioController.java* el método *registrar()* responde directamente con *usuarioGuardado*, es decir, la entidad completa.
- *GET /api/ranking/top-usuario* (mejor usuario del ranking): en *RankingController.java* el método *getTopUsuario()* responde con el objeto completo del usuario que tiene más puntos.

Consultamos el ranking sin ningún token y la respuesta trae el hash de la contraseña. Esto significa que cualquier persona que conozca la dirección de la API puede obtener los hashes de las cuentas y tratar de romperlos con calma. Aunque la cifra BCrypt dificulta el trabajo, exponer los hashes sigue siendo un riesgo grave. Además, la configuración de CORS definida en *Security.java* acepta cualquier origen (*) y permite el envío de credenciales, lo que agrava la situación.

El campo también queda expuesto en rutas autenticadas que devuelven la entidad, como *GET /api/usuarios/perfil*, por lo que el problema no se limita a las rutas públicas.

Recomendación: no devolver nunca el campo *contrasenna* en las respuestas, usando un DTO propio del usuario o marcando el campo con *@JsonIgnore*, y limitar el CORS a los orígenes reales de la aplicación.

4. H2 — Los puntos nunca se suman (severidad: alta)

El sistema de puntos y ranking no está funcionando. Revisando el código no encontramos ninguna operación que aumente los puntos de un estudiante. Lo único que hace el proyecto es crear los puntos en cero cuando alguien se registra (*UsuarioService.guardarUsuario()* hace *setPuntos(0)*), y volverlos a poner en cero cuando se cierra un periodo (*CierrePeriodoService*).

Esto significa que cuando un estudiante clasifica un material con la IA no gana nada, el ranking siempre mostrará a todos con cero puntos, y al cerrar el periodo el historial de ganadores queda guardado con puntos en cero también. Es decir, la parte principal del proyecto (la competencia por puntos) queda sin efecto.

Recomendación: agregar la operación que sume los puntos cuando se registra una clasificación y que ese aumento quede guardado en la base de datos (por ejemplo, *UPDATE usuarios SET puntos = puntos + X* al guardar un resultado de la IA).

5. H3 — El total de puntos del curso no se mantiene (severidad: media)

En la tabla *cursos* existe la columna *puntosTotales*, que en teoría debería ser la suma de los puntos de todos los estudiantes de ese curso. El problema es que ninguna parte del código actualiza esa columna cuando cambian los puntos: solo se puede modificar a mano desde el endpoint de actualizar curso (exclusivo de administradores) o se pone en cero con el cierre de periodo.

Es una falla de normalización, porque es un dato que se puede calcular a partir de los usuarios y en cambio está guardado aparte sin nada que lo mantenga al día. Como consecuencia, el ranking por cursos muestra información vieja o incorrecta.

Recomendación: calcular ese total en el momento con una consulta que sume los puntos por curso, o si se quiere conservar la columna, actualizarla dentro de la misma operación donde se otorgan los puntos.

6. Otros hallazgos menores

Además de los tres puntos anteriores detectamos varias cosas menores que también vale la pena mencionar:

- La clave con la que se firman los tokens JWT está escrita directamente en *application.properties* (*jwt.secret=TuClaveSuperSecretaDePruebaParaFirmarTokensJWT123456*), con un valor de prueba que es público. Si no se cambia, cualquiera podría armarse un token con rol de administrador.
- En el perfil de producción está activado *spring.jpa.hibernate.ddl-auto=update*, que permite que Hibernate modifique las tablas por su cuenta, algo que no es recomendable en un ambiente real.
- En el endpoint que guarda el resultado de la IA (*POST /api/registroia/guardar-resultado*) solo se validan la sesión y el material, pero no la categoría. Si llega sin categoría, el servidor responde con un error 500 en vez de un 400.
- El identificador de los periodos de ranking se genera con *Math.random()* (*PeriodoRanking.java*), así que en algún momento podrían chocar dos identificadores y fallar al guardar.

7. Conclusión

El repositorio de los compañeros sí se puede compilar, ejecutar y auditar, y tiene una base sólida sobre la cual trabajar. Sin embargo, encontramos un problema de seguridad grave (las contraseñas expuestas) que deberían corregir cuanto antes, un bug de lógica que deja sin funcionar todo el sistema de puntos, y una falla de normalización en el total de puntos por curso. Con esas correcciones el proyecto quedaría mucho más sólido para las siguientes entregas.